

AquaMeasure Guide du code

Ce guide présente l'organisation du dépôt, les principaux chemins de calcul et les vérifications à faire avant de livrer une modification. Les gestes dans l'application sont dans le [manuel utilisateur](#). Les formats de données et les calculs sont détaillés dans les [notes techniques](#).

Lancer et parcourir le projet

Le point d'entrée est `src/interface/main.py`. Suivre les commandes du [README](#) pour créer l'environnement `.venv` et installer `requirements.txt`. Python 3.10 et 3.11 conviennent à cette version. La distribution Windows a été vérifiée ; les autres systèmes demandent une validation des lecteurs vidéo et des moteurs IA.

Emplacement	Responsabilité
<code>src/interface</code>	Application PySide6, pages QML, contrôleurs et services.
<code>src/annotations</code>	Base SQLite, taxonomie, sessions, exports et outils de corpus.
<code>src/fish_detectors</code>	Catalogue des modèles et moteurs de détection.
<code>src/plugins/detectors</code>	Extensions de détection.
<code>src/aquameasure.py</code>	Synchronisation, calibration et vision réutilisées par l'interface.
<code>src/stereo_utils.py</code>	Correspondances et calculs stéréo.
<code>src/fish_annotate.py</code>	Façade des observations, pistes et événements.
<code>src/fishial_gallery.py</code>	Images validées et références locales de classification.
<code>tests</code>	Tests séparés de l'interface et des données.
<code>scripts et packaging</code>	Contrôles du dépôt, préparation et compilation Windows.
<code>docs</code>	Manuels, notes, captures, Word et PDF classés par thème.

Le module d'annotations était auparavant nommé `fish-vision`. Il reste utilisé par l'application actuelle. `src/aquameasure.py` contient aussi des classes historiques ; ses fonctions de vision sont importées par les services actuels. Le supprimer casserait la calibration et la mesure.

Le lanceur assemble le namespace Python `src` à partir de `src/interface/src` et `src/annotations/src`. Il ajoute également `src` au chemin des modules de vision. Ce montage explique les imports comme `src.controllers` et `src.annodb`. Utiliser le lanceur ou les scripts de test prévus pour retrouver ces chemins.

Interface et enregistrement

Dans `src/interface/qml`, `Main.qml` porte la fenêtre et la navigation. Les dossiers `pages`, `components` et `style` contiennent les sources de l'interface. Le sous-dossier `AquaMeasure` est généré au lancement ; ne pas le modifier ni le versionner.

Les contrôleurs Python de `src/interface/src/controllers` exposent des propriétés, signaux et actions à QML. Les services de backend exécutent les calculs. Une propriété informe l'interface, un Slot reçoit une action et un Signal annonce un changement.

`FishWorkflowPanel.qml` regroupe la fiche, la mesure, le suivi et les événements à gauche. `CompactRegistryPanel.qml` affiche le registre simple à droite. Le registre détaillé reste dans `Données & IA`. Les sections utilisent les composants communs et les couleurs de `Theme.qml`.

Le clic droit passe par `Data.selectBoxExplicit`. `Data.saveFish` crée l'observation si elle est nouvelle, sinon met à jour celle qui est sélectionnée. Le bouton Enregistrer le poisson assure les deux cas. Une correction d'espèce conserve l'identifiant et la provenance du modèle.

Après une sauvegarde réussie, les points A et B de travail disparaissent des vues ; la longueur enregistrée reste dans la fiche. En cas d'échec, les points sont conservés pour réessayer. Ne pas recharger la calibration au milieu de cette action : cela peut relancer la détection et invalider le cadre sélectionné.

Synchronisation calibration et mesure

La synchronisation calcule un décalage entre les vidéos, par flash ou par repères manuels. Chaque prise conserve son décalage. Les bornes de découpe vidéo et les bornes In et Out d'une piste servent à des opérations différentes.

Le service de calibration appelle le moteur ChArUco de `src/aquameasure.py`. Le scan détecte les coins, puis le calcul estime les paramètres de chaque caméra et leur géométrie relative. La taille réelle des cases fixe l'échelle. Lire la [note de calibration](#) pour les paramètres et les contrôles.

La mesure triangule deux points anatomiques vus dans les deux caméras. Leur distance fournit la longueur en millimètres. `MeasureController` pilote ce parcours avec les services stéréo. La [note de mesure](#) explique la disparité et les sources d'erreur. Une boîte de détection situe le poisson ; elle ne définit pas sa longueur anatomique.

Les coordonnées raw appartiennent à la vidéo brute ; `stereo_rectified_left` désigne l'image gauche rectifiée. `RectMapping`, dans `src/interface/src/imaging`, convertit les repères entre ces images. `MeasureStereoView.qml` applique ensuite le même zoom et le même déplacement à l'image, aux cadres et aux marqueurs. Vérifier la pause et la lecture.

Suivi et événements

`FishController.beginTrackFollow` et `finishTrackFollow` produisent une trajectoire et la rattachent au poisson, sans comportement implicite. Une observation enregistrée et un cadre localisé sont nécessaires à l'amorce ; la longueur reste facultative. Le parcours courant pose In sur l'image de la fiche avant de naviguer vers Out.

`beginBehaviorFollow` retient le type choisi au départ. Après un rattachement réussi, `Data.completeAssistedBehavior` enregistre la durée. Si le poisson porte déjà une piste, le mode durée utilise celle-ci sans recalculer son déplacement. Un conflit de rattachement empêche l'écriture de l'intervalle.

Le suivi utilise Ultralytics et ByteTrack. Si le détecteur sélectionné ne propose pas cette intégration, le logiciel cherche un modèle Ultralytics installé. Un croisement ou une occultation peut faire perdre le poisson ; l'identifiant ne garantit pas la justesse de toutes les positions.

`FishFollowPanel.qml` distingue Comportement à cet instant, Comportement sur une durée et Trajectoire. `TrackPeckPanel.qml` porte la commande de pointage. Sans piste, elle appelle `Data.toggleSelectedPointEvent` sur l'image de l'observation ; avec une piste, elle appelle `PeckController` sur l'image courante.

`PeckController` gère ajout, retrait, navigation et relecture des points. `TrackEventList.qml` liste les événements. `EventMarkerStrip.qml` place les repères temporels. Le nom validé est propagé aux overlays par `fish_annotate.list_video_annotation_overlays` et le cache de `FishController`.

Les types d'événement se créent dans les préférences via `Data.addBehaviorType`, avec portée `instant` ou `interval`. Bouchée est ponctuel par défaut. Un nouveau type utilisateur ne demande aucun code ; pour proposer un symbole supplémentaire, modifier `behaviorLogos` dans `SettingsPage.qml`.

Données et modèles

Le schéma ORM est dans `src/annotations/src/annodb/models.py`. `connection.py` initialise et migre les bases. Les vidéos sont des fichiers externes. La [note de données](#) décrit les tables, les images absolues et les coordonnées.

La calibration liée à une prise est une référence de provenance. Réouvrir la session ne réimporte pas son ancien ZIP : la mesure utilise les fichiers actifs de `camera_parameters`. Si la calibration active a changé, le parcours demande de réimporter la bonne calibration.

Une migration doit conserver les anciennes bases et accepter une seconde exécution. Les index historiques demandent l'offset figé de leur prise. Sans conversion fiable, conserver l'information source et signaler la limite ; ne pas inventer un numéro d'image ou une valeur manquante.

La [note d'export](#) décrit le paquet CSV complet et les événements COCO-VID. Ajouter un champ en base ne l'ajoute pas aux exports : adapter chaque format concerné et son dictionnaire.

La bibliothèque Fishial réunit les observations validées de toutes les sessions locales. L'ajout de références calcule les nouveaux vecteurs avec seuil et déduplication. Le transfert passe par `annodb/fishial_transfer.py` ; il contrôle le modèle, les dimensions et les empreintes avant de fusionner les références.

Pour des poids compatibles, utiliser le gestionnaire de modèles. Un moteur nouveau implémente `DetectorBackend` avec `load`, `infer` et `unload`. L'inférence renvoie les coordonnées de boîte, confiance, identifiant et nom de classe. Voir le [guide des modèles](#) pour les plugins et dépendances. Ajouter une architecture Python absente d'un exécutable demande une nouvelle compilation.

Détection, suivi, calibration et exports utilisent les workers existants. Un résultat reçu doit encore correspondre à la vidéo, à l'image et à la sélection actives. Conserver l'invalidation des caches et arrêter proprement les workers à la fermeture.

Vérifier une modification

Ces commandes se lancent à la racine du dépôt. Le script configure les chemins Python et des données temporaires pour protéger la base du poste.

```
.\.env\Scripts\python.exe scripts/run_tests.py --core
.\.env\Scripts\python.exe scripts/run_tests.py --suite data
.\.env\Scripts\python.exe scripts/run_tests.py --suite interface --pattern
test_measure_qml_runtime.py
```

--core lance les contrôles de taxonomie, sessions, CSV et événements COCO-VID utilisés par GitHub Actions. --pattern cible une suite. Installer les dépendances des moteurs avant de lancer leurs tests.

Après une modification du suivi, contrôler une sélection à la pause, un zoom, le passage en lecture et la relecture d'un événement. Pour le packaging, exécuter aussi le binaire et une installation isolée. Un test Python seul ne vérifie pas la présence d'une DLL dans un installateur.

Compiler sous Windows

La compilation complète attend les poids de `packaging/models-manifest.json`. Les fichiers internes `fish_detect_public` et `fish_detect_family` ne sont pas fournis sur GitHub. Pour une autre sélection, adapter `scripts/prepare_release_models.py`, le manifeste et les contrôles de livraison. Les [licences des composants](#) s'appliquent à la redistribution.

1. Installer les dépendances de l'application et celles de `packaging/requirements-windows-full.txt` dans `.venv`.
2. Préparer les poids requis dans `src/annotations/models`, puis exécuter `scripts/prepare_release_models.py`.
3. Vérifier les manuels dans `docs/word` et `docs/pdf`.
4. Exécuter `scripts/build_windows.ps1 -SkipArchive`. Le script utilise PyTorch CPU, prépare le QML et appelle `packaging/aquameasure-windows.spec`.
5. Construire l'installateur avec `scripts/build_installer.ps1` à partir du dossier portable produit.
6. Lancer les autotests du binaire et `scripts/smoke_test_windows_installer.ps1` dans une installation isolée.

Le binaire range les fichiers de `src` dans son propre dossier. Les données de session, vidéos, calibrations personnelles et jetons sont exclus. Les noms taxonomiques sont livrés ; les références Fishial personnelles passent par l'export de bibliothèque.

Le logo IRD est dans `src/interface/resources`. L'ICO sert à Qt, PyInstaller et Inno Setup. Sans certificat de signature de code, le résultat reste non signé. Une empreinte SHA-256 vérifie le fichier reçu ; elle ne remplace pas une signature d'éditeur.

Entretenir la documentation

`docs/documents.json` liste les huit guides, leur source Markdown et leur catégorie. `python docs/build_word.py` régénère les Word. `scripts/render_documentation.ps1` produit les PDF avec Word sous Windows. Les deux arborescences utilisent les mêmes catégories.

`docs/capture_screenshots.py` ouvre la vraie interface avec une base isolée. Les options `--source-db`, `--session-id` et `--data-root` permettent de travailler sur une copie de session. Le mode `--onscreen` affiche les lecteurs natifs nécessaires aux captures vidéo. Relire les libellés et contrôler les images avant de les intégrer au manuel.

`scripts/check_publication.py` vérifie les fichiers suivis et les liens. Publier les sources et documents courants ; les binaires, modèles, données et archives anciennes restent hors du dépôt.